

CIS 206 - Assignment 5 Testing Plan and Experience

Christian “CJ” Valle

Testing within the development of a program is more complex than I originally thought, tools and ease-of-use features and entire libraries are developed just for the purpose of assisting developers within the testing of their program.

Testing technology and the debugging that comes with it

Honestly, I planned to take the easiest way out and use the pre-installed **unittest**. However, while sorting through resources it became apparent that **pytest** was much more useful and preferred, so I ended up swapping to it. Though, this came at great struggle, as I spent quite a while debugging pytest to ensure it was installed correctly on my system.

(Apparently Python’s install alias may be **py** instead of **python** based on how you obtained it.. That was fun to figure out..)

```
C:\Users\cjval>pip pytest --version
ERROR: unknown command "pytest"

C:\Users\cjval>pytest --version
'pytest' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\cjval>python -m pytest --version
The system cannot find the file C:\Users\cjval\AppData\Local\Microsoft\WindowsApps\python.exe.

C:\Users\cjval>python -m pip show pytest
The system cannot find the file C:\Users\cjval\AppData\Local\Microsoft\WindowsApps\python.exe.

C:\Users\cjval>python -m pip show pytest
The system cannot find the file C:\Users\cjval\AppData\Local\Microsoft\WindowsApps\python.exe.

C:\Users\cjval>python -m pip install pytest
The system cannot find the file C:\Users\cjval\AppData\Local\Microsoft\WindowsApps\python.exe.

C:\Users\cjval>where python
C:\Users\cjval\AppData\Local\Microsoft\WindowsApps\python.exe

C:\Users\cjval>python -m pip show pytest
The system cannot find the file C:\Users\cjval\AppData\Local\Microsoft\WindowsApps\python.exe.
```

A little hint of what I was dealing with. I was very shocked when my system both found, and couldn't find python.

Putting it in practice

Moving onto the testing phase, there were lots of resources that assisted me in setting up pytest properly. At first I wasn't sure if it only took algebraic functions, luckily there was more that you could do with the testing infrastructure that allowed for me to run other types of tests. I kept everything rather simple though, taking my functions and splitting them into separate tests.

test_height_process simulates feet and inches entered by a user, with an expected value of 70 based on how the math works.

Similarly, **test_bmi_process** takes weight and height and uses them to calculate BMI. The values here are in an attempt to get a nice number, but since I did approximations anyways, this could be replaced with anything else.

report is the one that got split into three values, the tests were conducted by plugging a value in that corresponds to each of the ranges, then making sure the output is valid, thus creating **test_report_under**, **test_report_normal** and **test_report_over**.

```
3 import pytest
4
5 from Assignment_5 import height_process, bmi_process, report_process
6
7
8 def test_height_process():
9     assert height_process(5, 10) == 70
10
11
12 def test_bmi_process():
13     assert bmi_process(130, 66) == pytest.approx(21, abs=.1)
14
15
16 def test_report_under():
17     assert report_process(18) == "Underweight"
18
19
20 def test_report_normal():
21     assert report_process(22) == "within the normal range"
22
23
24 def test_report_over():
25     assert report_process(28) == "Overweight"
```

✓ Assignment 5 (5)	1 ms
✓ test_functions.py (5)	1 ms
✓ test_functions (5)	1 ms
✓ test_bmi_process	1 ms
✓ test_height_process	< 1 ms
✓ test_report_normal	< 1 ms
✓ test_report_over	< 1 ms
✓ test_report_under	< 1 ms

These here are the completed and validated tests, but this was not my first version of the tests. My first rounds looked like this instead;

✗ Assignment 5 (5)	2 ms	
✗ test_functions.py (5)	2 ms	
✗ test_functions (5)	2 ms	
✗ test_bmi_process	1 ms	assert 20.980...
✓ test_height_process	< 1 ms	
✓ test_report_normal	< 1 ms	
✗ test_report_over	1 ms	AssertionErro...
✓ test_report_under	< 1 ms	

(This is an attempt to recreate what went wrong, as I am writing this paper a day after I set up the code)

For **test_report_over**, this break was actually expected, as I put in a value of 0 to ensure that it would fail. However, I was blindsided by **test_bmi_process** failing. After some research, I found out that due to how **BMI** was calculated that I would have to do some rounding and approximation since I did not want to write out a long decimal, so I went with an absolute value in **pytest**. Upon attempting to use *abs* for the first time, I was met with this when I tried to rerun the tests.

Build your solution to discover all available tests. Click "Run All" to build, discover, and run all tests in your solution.

Note: Tests in UWP projects can only be discovered by clicking "Run All".

While the intent was pure of heart, and logically it would make sense. Much to dismay though, there was an issue I had not foreseen, one that made me question the installation of python and **pytest** once again. This took me about half an hour to figure out, I was on the brink of giving up for the day until a friend mentioned that maybe I had forgotten a semicolon somewhere, as a joke. As it would turn out, the issue was very similar.

```
def test_bmi_process():  
    assert bmi_process(130, 66) == pytest.approx(21, abs=.1)
```

I had forgotten to insert an equal sign between abs and .1. This single typo was responsible for an immense struggle, and it really shows how even the smallest oversight can hinder an entire program and why tests are so important, to catch bugs, errors and even typos like this.